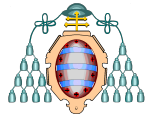




Apellidos:

Nombre:

D.N.I.:



INTRODUCCIÓN A LA PROGRAMACIÓN - E.P. DE INGENIERÍA DE GIJÓN

15 de Enero de 2024

1. (2 %) Declara un vector de cadenas de caracteres donde se podrán almacenar 10 cadenas.

2. (5 %) Dada la siguiente definición de variables y sus valores iniciales,

```
int a=6, b=4; double f=5.0; char c='a'; String s="hola";
```

indica para las siguientes expresiones, si son o no correctas (SI/NO), en caso de resultar incorrectas JUSTIFICA por qué lo son, y en el caso de ser correctas indica el TIPO y el VALOR que producen.

		Tipo	Valor	Motivo
<code>s[3] == c</code>	☐ Si ☐ No			
<code>(char)(c+b)</code>	☐ Si ☐ No			
<code>a/b+f</code>	☐ Si ☐ No			
<code>(int)f / f</code>	☐ Si ☐ No			
<code>b**2</code>	☐ Si ☐ No			

3. (9 %) Dado el string `s="Pienso, luego existo."`, indica lo que imprimen los siguientes esquemas iterativos.

<pre>int i=1; while (i<s.length()){ System.out.print(s.charAt(i)); i+=5; }</pre>	<pre>for(int j=6; j>=0;j--){ System.out.print(s.charAt(j)); }</pre>	<pre>int i=0; while(i<s.length()){ if(s.charAt(i) == 'o') System.out.print('*'); i++; }</pre>

4. (10 %) Diseñar el método público y estático `mediaDeTernas` que recibe un vector de números enteros y retorna un vector de números reales donde se almacenan las medias de cada tres valores del vector que se pasa como parámetro (cuyo número de elementos siempre será múltiplo de 3). Ejemplo: vector inicial {6,9,3,7,4,5,1,8,2}, vector resultado {6.00, 5.33, 3.67}.

5. (6 %) Dadas dos variables enteras x e y escribe las siguientes condiciones:

Que x sea múltiplo de y y que al menos una de las dos sea divisor de 100:

Que x contenga un valor válido como índice de un vector de enteros v y que el valor en la posición x sea igual a y :

Que x e y sean del mismo signo:

6. (12 %) En un kiosko hacen descuentos a los menores en función de la categoría del menor y del número de gominolas compradas. Se pide hacer un **programa de consola completo (importando las clases necesarias)** que pida por teclado la categoría (que se introduce mediante la primera letra) y el número de gominolas compradas, y que muestre en la consola el porcentaje de descuento que se aplicará escribiendo un mensaje como el siguiente: **[Categoría=A, Unidades=13] Descuento: 25%**, si la categoría es alevín y compra 13 unidades.

	2	[3,5]	[6,20]	21 o más
(I)nfantil	10	10	20	30
(A)levín	10	10	25	30
(C)adete	20	30	35	35
(J)uvenil	25	30	35	35

7. (8 %) Escribe un trozo de programa en el que ya disponemos de un **String** llamado **s** que contiene un texto (en minúsculas) y se quiere mostrar en la consola si en **s** está presente la conjunción y. Se considera que la conjunción está presente si aparece la letra 'y' rodeada de espacios en blanco.

Ejemplos. “blanco y negro” → sí está, ”está ya cansado” → no está.

8. (8 %) Escribe un método público y estático llamado **conjunciónY** que reciba un **String** y detecte lo mismo que el ejercicio anterior. El método debe devolver **true** si la conjunción y está presente y **false** en caso contrario.

9. (15 %) Escribir el método público y estático **cadenaColumna** que recibe una matriz de caracteres y un número entero (que indicará la columna). El método debe retornar un **String** con los caracteres que aparezcan en la columna indicada como parámetro.

Matriz m			
a	b	c	d
e	f	g	h
i	j	k	l
m	n	ñ	o

⇒

Ejemplos	
llamada:	cadena de retorno:
<code>cadenaColumna(m,0):</code>	”aeim”
<code>cadenaColumna(m,2):</code>	”cgkñ”

10. (20 %) Implementa la clase **Furgoneta** que permite almacenar la capacidad de carga de distintas furgonetas. La información que debe recoger cada objeto de esta clase consiste en el nombre del modelo de la furgoneta, la capacidad de carga en m^3 y el porcentaje de espacio ocupado.

- Constructores: por defecto (con los valores "Kangoo" para nombre del modelo, 3.3 para la capacidad y 0 para porcentaje de espacio ocupado), el de copia y otro con 3 parámetros que se corresponderán con modelo, capacidad y ocupación.
- Métodos `get()` y `set()`. La capacidad tiene que ser mayor que 0 y la ocupación debe ser un valor entre 0 y 100.
- Método `calculaEspacioLibre()`, que calcula los m^3 libres.
- Método `tieneMásEspacioLibre()` que recibe un objeto de la clase **Furgoneta** y devuelve cierto si la furgoneta con la que se llama al método tiene más espacio libre que la furgoneta que se pasa como parámetro, y falso en caso contrario.
- Método `toString()`, que devuelve un String con los datos del objeto **Furgoneta** con el siguiente formato: "(Caddy, 3.7, 11.8)" (si la furgoneta fuese una Caddy con 3.7 m^3 de capacidad y ocupada en un 11.8%).

11. (5 %) Hacer un programa de prueba de la clase **Furgoneta**. El programa debe crear varios objetos usando todos los constructores y emplear esos objetos para probar los métodos **tieneMásEspacioLibre()** y **toString()**.